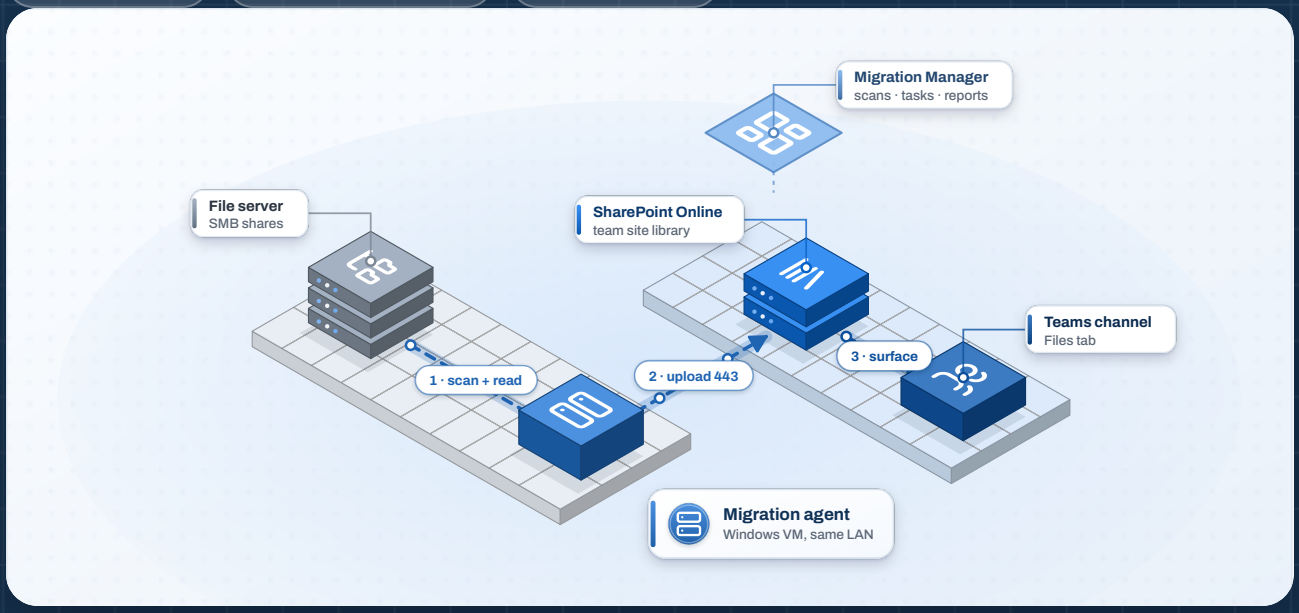
STANDARD OPERATING PROCEDURE · FIELD GUIDE SERIES

SOP-M365-001

Migrating On-Premises File Shares to Microsoft Teams and SharePoint

Scan, remediate, migrate and cut over Windows file shares into Teams channels with Migration Manager, the SharePoint Migration Tool (GUI and PowerShell), and a scripted pre-migration cleanup.

Microsoft 365 | SharePoint & Teams | Data Migration



DOCUMENT ID	SOP-M365-001
VERSION	2.0
EFFECTIVE	October 2026
AUDIENCE	Microsoft 365 administrators, systems engineers, IT project leads
AUTHOR	Richmond Dieujuste, Principal Consultant

Rich Dieu IT Solutions LLC · richdieu.com

Sanitized reference edition — client identifiers removed

Contents

1. Concepts You Need First 3

1.1 How Teams files live in SharePoint 3

1.2 Migration Manager architecture 4

1.3 SharePoint naming rules and limits 5

1.4 Permissions mapping 6

1.5 Choosing a method 7

1.6 Delta migration and cutover 8

2. Access, Prerequisites and Planning 10

2.1 Level of access required 10

2.2 Prerequisites checklist 11

3. Method A · Migration Manager in the SharePoint Admin Center 13

3.1 Remediating scan warnings 15

4. Methods B, C and D · SPMT and the Sync Alternative 20

4.1 Method D · Copying through OneDrive sync (not recommended) 23

5. Cutover, Verification and Recovery 24

5.1 Pilot 24

5.2 Cutover runbook 24

5.3 Verification and evidence 25

5.4 Rollback 25

5.5 Troubleshooting 26

6. Summary, Glossary and Review 28

6.1 Chapter summary 28

6.2 Glossary 28

LEARNING OBJECTIVES

After working through this procedure you will be able to:

- Explain how Teams channel files are stored in SharePoint document libraries and what that means for permissions and URLs.
- Describe the Migration Manager architecture: agents, scans, tasks and reports.

- Identify and remediate SharePoint blockers: invalid names, empty files, long paths and unsupported items.
- Migrate shares into Teams with Migration Manager, the SPMT desktop tool and an SPMT PowerShell template driven by a CSV.
- Plan delta (incremental) passes and a clean cutover that leaves the old share read-only.
- Assign least-privilege roles and produce evidence and rollback steps for the change record.

TIME	DIFFICULTY	MINIMUM ROLE	RISK TO SOURCE DATA
Pilot ≈ 1 day; bulk depends on data volume	Intermediate	SharePoint Administrator + Read on source shares	Low (copy only; cleanup step deletes junk files)

CHAPTER 1

Concepts You Need First

Moving a department off a Windows file server and into Microsoft Teams looks like a copy job: files in, files out. In practice it is a change of *storage platform*. An SMB file share is a hierarchical file system governed by NTFS permissions. Teams stores files in **SharePoint Online**, which is a web-based document management system with its own naming rules, size and path limits, permission model and metadata. Most migration problems come from the gap between those two worlds, so this chapter explains it before any button is clicked.

> **KEY CONCEPT · TEAMS IS THE FRONT DOOR, SHAREPOINT IS THE WAREHOUSE**
>
> When you open the `Files` tab of a Teams channel you are looking through a window into a SharePoint document library. Teams itself stores no files. Think of Teams as the shop counter where people meet and talk, and SharePoint as the warehouse behind it where the stock actually sits. A migration tool therefore never "uploads to Teams". It writes into the warehouse, into the right aisle (the channel folder), and Teams shows the result.

1.1 How Teams files live in SharePoint

Every team is backed by a **Microsoft 365 group**. The group owns a **SharePoint team site**, and the site has a document library called *Documents* (its URL segment is `Shared Documents`). Each **standard channel** is simply a top-level folder in that library with the same name as the channel. **Private and shared channels** are different: each gets its *own* SharePoint site with its own membership, which is why they are the right home for content that only part of the team may see.

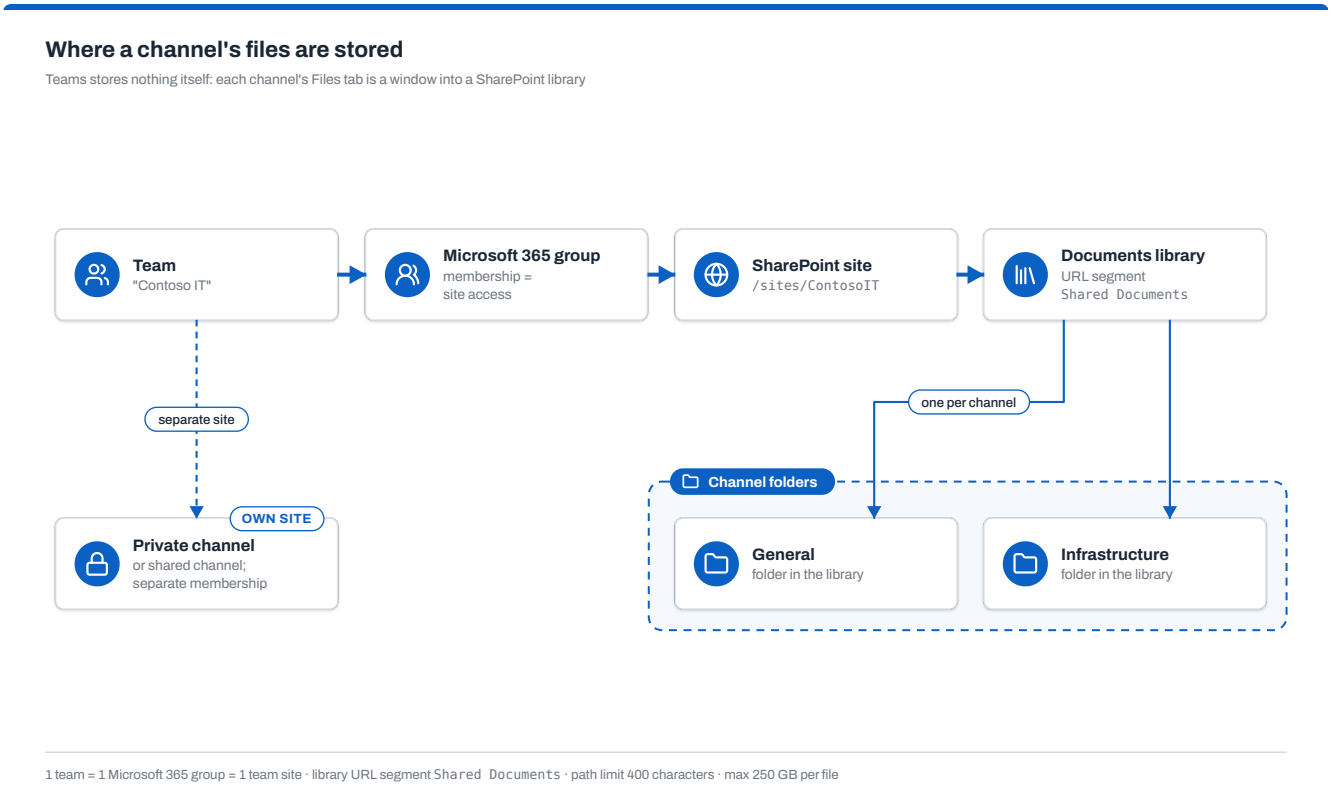


Figure 1. Because a standard channel is only a folder in the team's library, access follows team membership; content that needs a smaller audience belongs in a private channel, which gets its own site.

Three consequences follow, and each one shapes a later step:

- **Permissions come from team membership.** Members of the team are members of the site. Recreating the old folder-by-folder NTFS permissions inside a channel folder works against this model (see *Permissions mapping* below).
- **The destination URL is long before your first file lands.** `https://contoso.sharepoint.com/sites/ContosoIT/Shared Documents/Infrastructure/` is already about 80 characters, and it counts toward SharePoint's path limit.

1.2 Migration Manager architecture

Migration Manager is Microsoft's free, cloud-managed migration service in the SharePoint admin center. It separates *control* from *work*:

- The **control plane** is the SharePoint admin center › Migration › File shares page. Administrators create scans and tasks there, watch progress and download reports.
- The **agent** is a small Windows service installed on one or more on-premises Windows machines (typically a dedicated Windows Server VM close to the file server). It runs under a Windows account that can read the shares, polls the cloud for work over HTTPS 443, reads the files over SMB, packages them and uploads them to Microsoft-provided Azure storage, from which SharePoint imports them through the *Migration API*.

- **Agent groups** let you add several agents. Tasks are distributed across the agents in a group, which is how large migrations are parallelised.

WHY IT MATTERS · AGENT PLACEMENT

The agent reads every byte over SMB and then uploads it. Put it on the same LAN segment as the file server, never across a slow WAN link, and give it enough free disk for temporary packages. Its upload bandwidth to the internet is usually the real bottleneck.

Two objects do the work. A **scan** walks a source path and reports what *would* happen: total size, item counts and every item that will fail or be modified. A **task** (on the *Migrations* tab) is the actual copy of one source to one destination. A task can be re-run; re-running copies only what changed, which is the basis of delta migration.

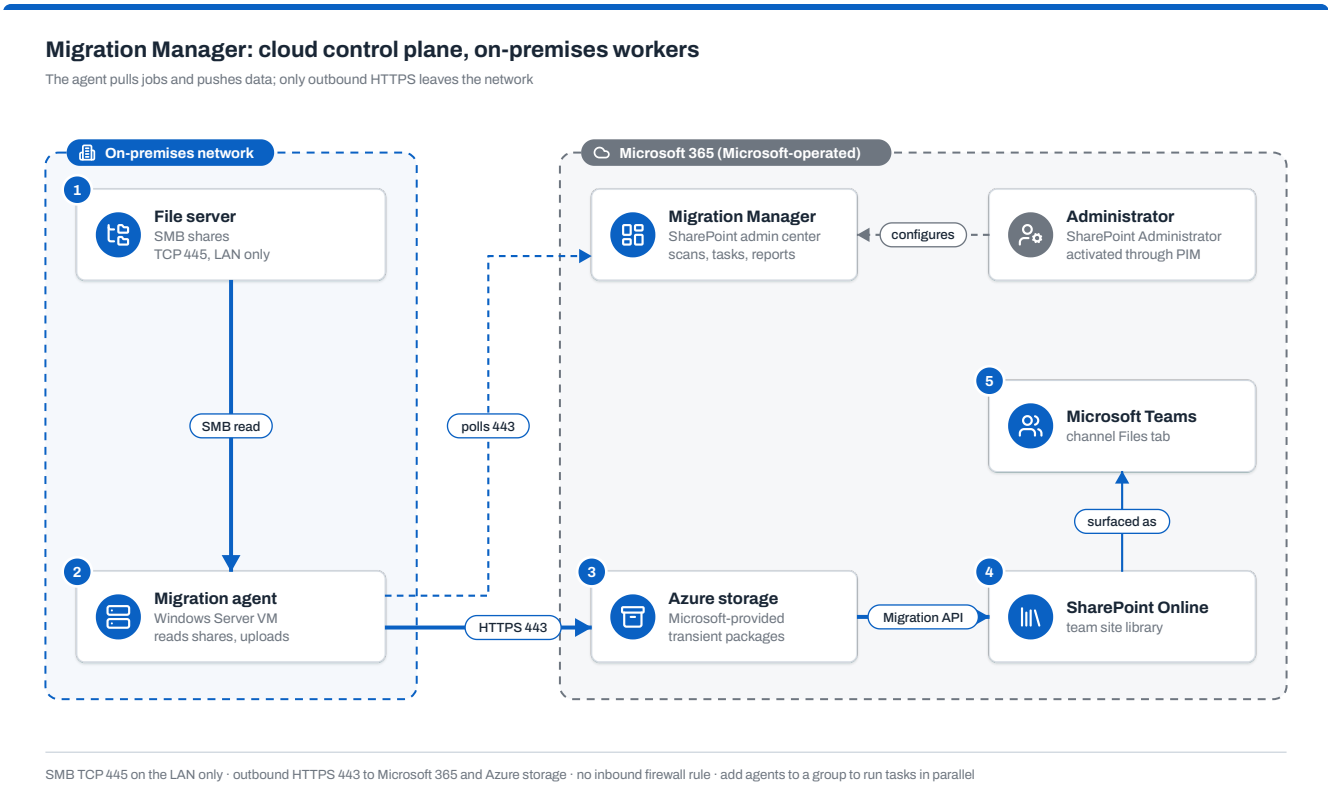


Figure 2. The agent initiates every connection, so the file server never needs to be reachable from the internet and no inbound firewall rule is required.

1.3 SharePoint naming rules and limits

NTFS already forbids `\ / : * ? " < > |` in names, so most files arriving from a Windows share pass the character test. What does not pass are names that SharePoint reserves or treats as system files. The scan log reports these as `INVALID_SHAREPOINT_NAME`.

Rule	Detail and typical offenders
Blocked names	`.lock`, `CON`, `PRN`, `AUX`, `NUL`, `COM0`–`COM9`, `LPT0`–`LPT9`, `desktop.ini`, and any name containing `_vti_`.
Blocked prefixes	Names that begin with `~$` (Office owner/lock files left behind when a document was open).
Leading / trailing spaces	Not allowed in file or folder names; usually created by Mac clients or scripts.
Path length	The decoded path, including site URL, library, folders and file name, may not exceed **400 characters**. A channel prefix such as `https://contoso.sharepoint.com/sites/ContosoIT/Shared Documents/Infrastructure/Projects/` already uses 87 of them.
File size	Maximum 250 GB per file.

Empty and junk files. Zero-byte files (`ITEM_IS_EMPTY`) are almost always debris: failed saves, placeholder files from old applications or sync conflicts. Hidden system files such as `Thumbs.db`, `desktop.ini` and `~$` owner files only make sense on a local file system. Removing them before migration produces a clean scan and a smaller failure report, which is far easier to sign off.

1.4 Permissions mapping

Both Migration Manager and SPMT offer **Preserve file share permissions**. When enabled, the tool reads each item's NTFS access control list, looks up each Active Directory identity in Microsoft Entra ID (it must be synchronised by Entra Connect), and writes item-level SharePoint permissions. That sounds safe, but it creates thousands of *unique permissions* inside a library whose whole design is "the team can see the team's files".

> **WATCH OUT · DO NOT DRAG BROKEN INHERITANCE INTO TEAMS**
>
> The recommended pattern is to **leave permission preservation off** and re-express access in Teams terms: who is in the team, which content needs a private channel, and which content belongs in a separate team. Use preservation only for a lift-and-shift into a plain SharePoint site where the business explicitly requires the old ACLs, and document why.

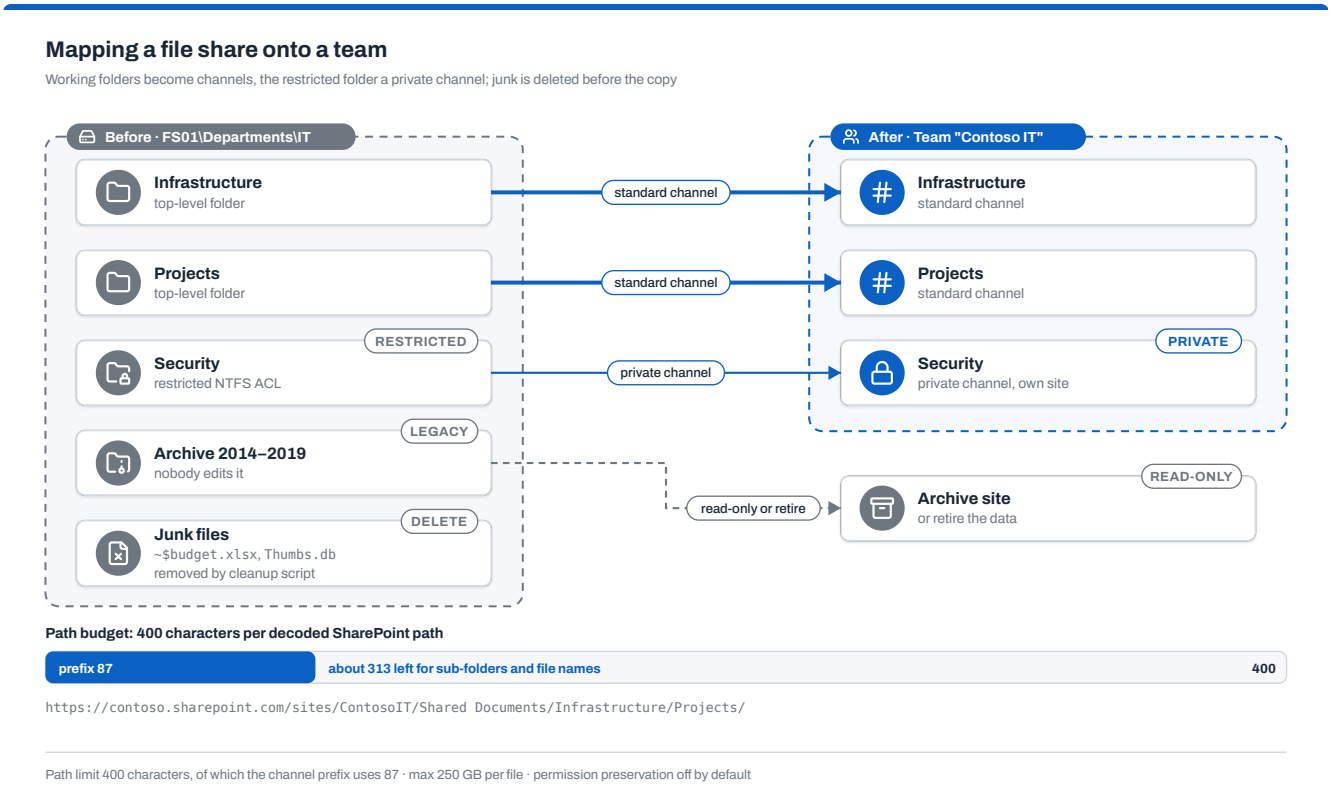


Figure 3. Re-express the old folder ACLs as team, channel and private-channel membership instead of copying them, and budget folder depth early: the channel prefix alone uses about a fifth of the 400-character path limit.

1.5 Choosing a method

A · Migration Manager

Cloud-managed in the SharePoint admin center, agents on-premises, parallel and centrally reported.

BEST FOR: WHOLE DEPARTMENTS, MANY SHARES

B · SPMT desktop

The SharePoint Migration Tool installed on one Windows machine; wizard-driven.

BEST FOR: A FEW SHARES, SMALL TEAMS

C · SPMT PowerShell + CSV

Scripted, repeatable migrations from a source → destination mapping file.

BEST FOR: WAVES, SCHEDULED DELTAS, CHANGE CONTROL

D · Copy via OneDrive sync

Robocopy into a synced library folder. Loses metadata and has no reporting.

BEST FOR: ALMOST NOTHING; CONTRAST ONLY

Whichever method copies the data, the **pre-migration cleanup script** in Chapter 3 is used to remediate the scan warnings.

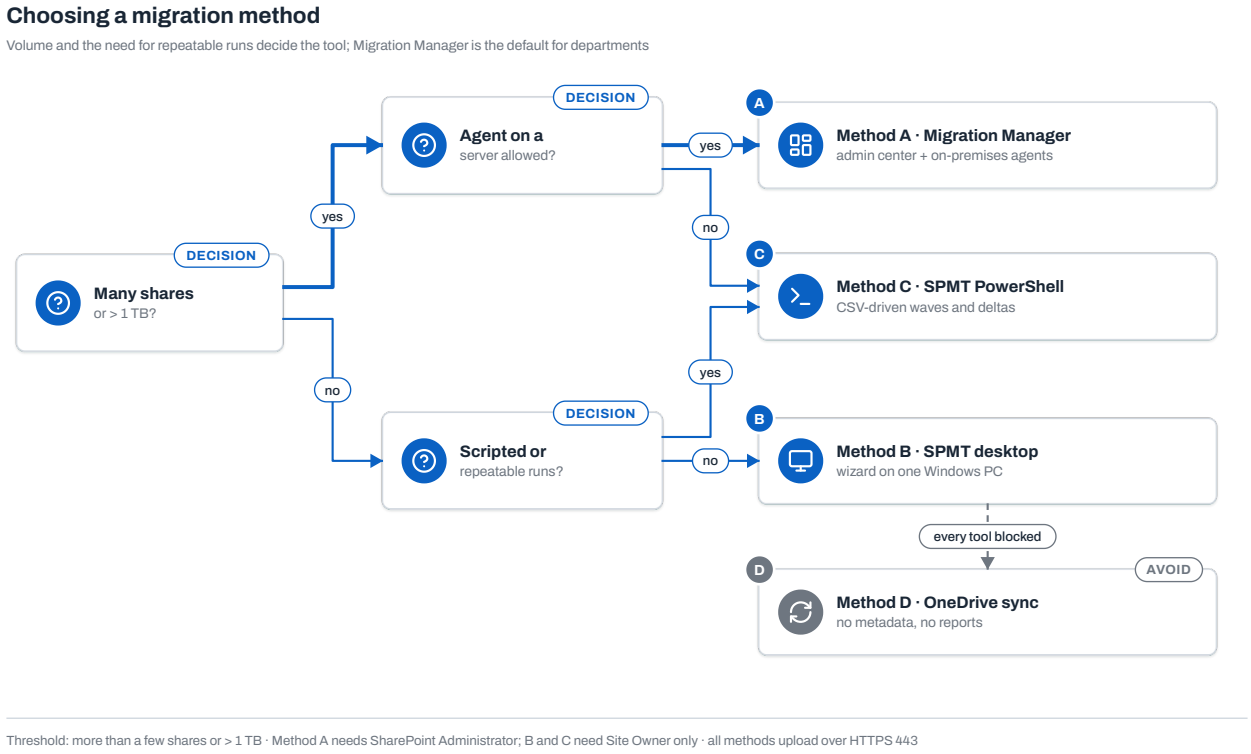


Figure 4. Pick the tool from volume and repeatability; OneDrive sync is a last resort because it loses metadata and leaves no migration report.

1.6 Delta migration and cutover

Large shares cannot be copied in a single outage. Instead you copy the bulk while users keep working, then run **delta** (incremental) passes that copy only files created or changed since the previous pass. Each pass is smaller than the last. At **cutover** the source is made read-only, one final delta runs, and users are pointed at Teams. Note that deltas copy new and changed files but do *not* delete files at the destination that users deleted at the source; keep the source read-only during the final pass to avoid surprises.

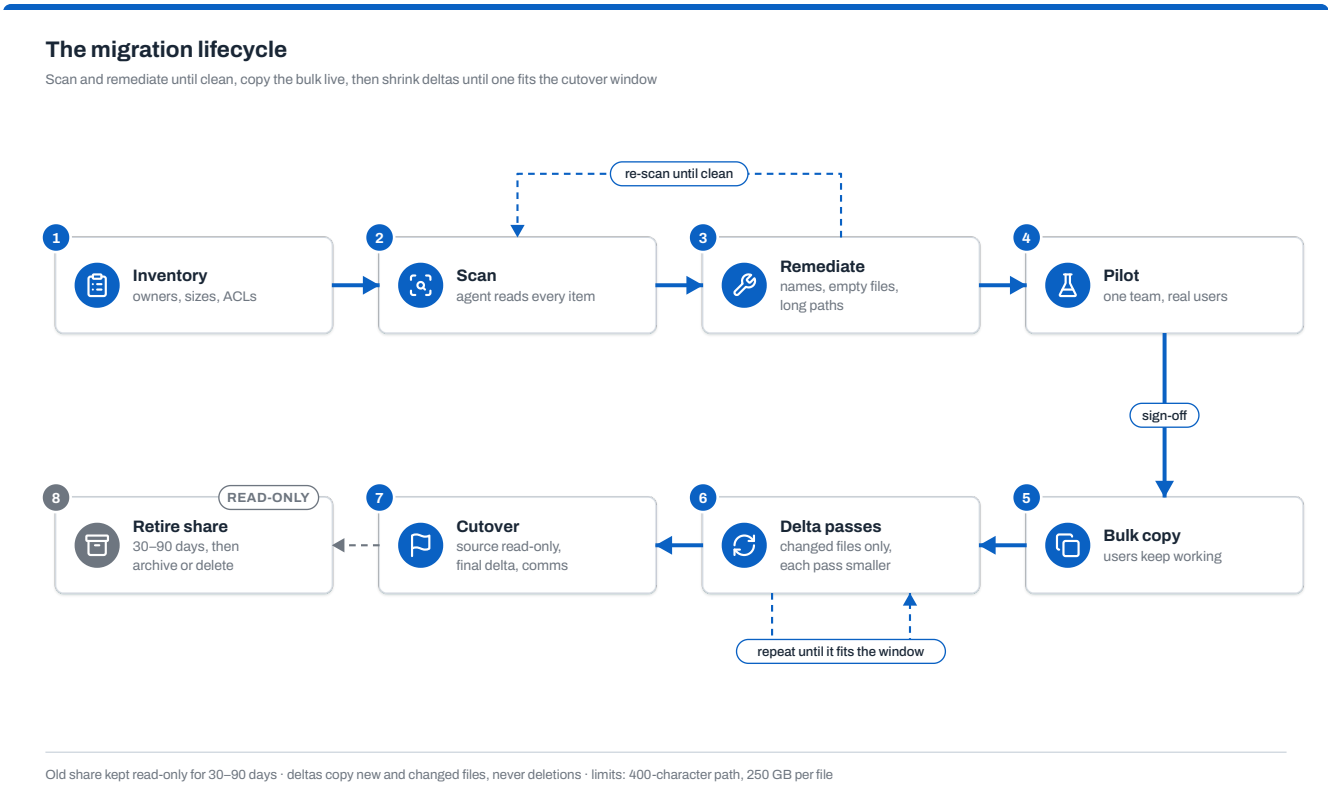


Figure 5. Remediation repeats until the scan is clean and delta passes repeat until the last one fits inside the cutover window; after cutover the old share stays read-only for 30 to 90 days.

CHAPTER 2

Access, Prerequisites and Planning

2.1 Level of access required

A file share migration crosses two security boundaries: Microsoft 365 roles in the cloud and Windows permissions on-premises. The agent's Windows account must be able to *read* everything you migrate; if it cannot read a folder, that folder silently becomes a failure line in a report. Grant each role at the narrowest scope that works.

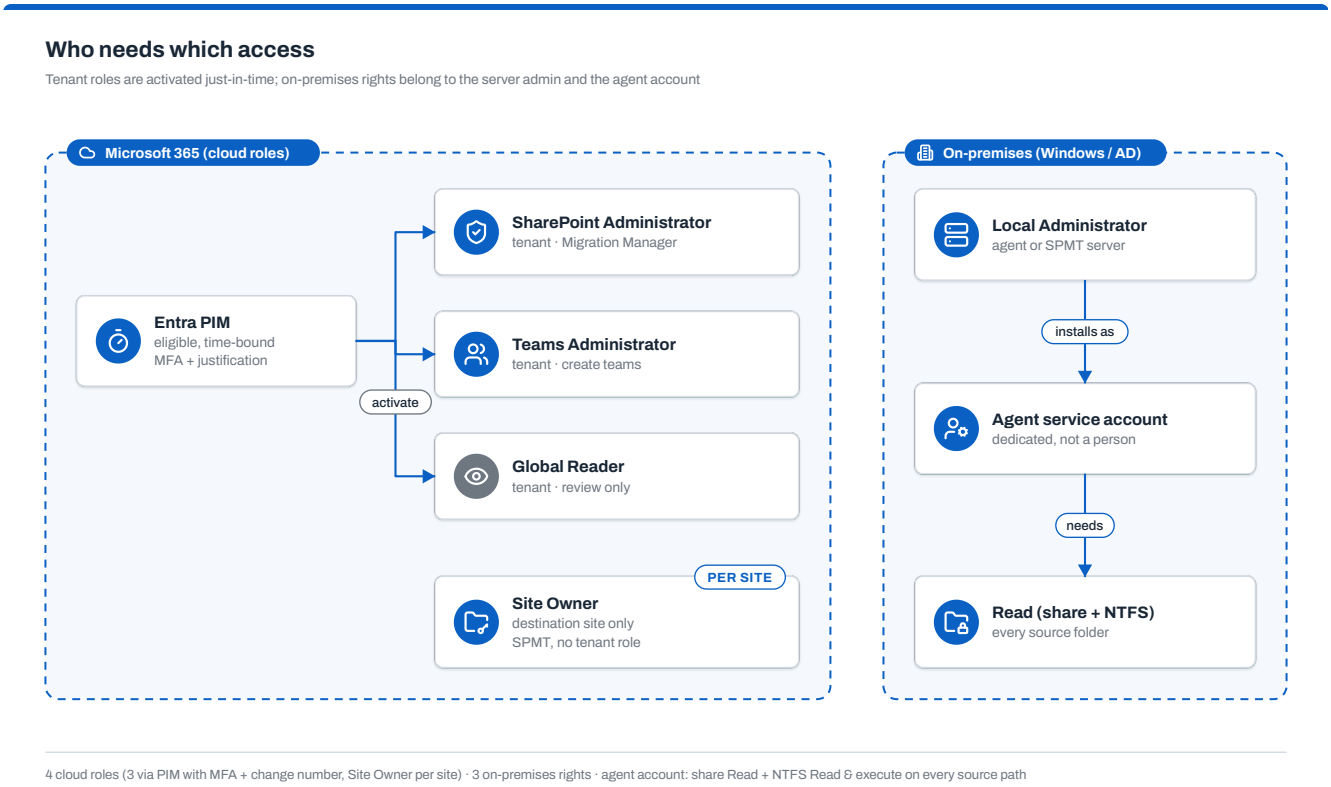


Figure 6. Cloud roles are held only for the migration window through PIM, while the agent's dedicated service account carries the on-premises Read that every source path needs.

Task	Least-privilege role / right	Scope and notes
Open Migration Manager, create scans and tasks, register agents	SharePoint Administrator (Entra ID role)	Tenant. Global Administrator also works but is excessive. The agent is registered by signing in with this account.
Create the destination team and channels	Teams Administrator, or team Owner for channels	Tenant / team. Can also be done by the business owner if self-service team creation is allowed.

Review settings and reports only	Global Reader	Tenant, read-only. Suitable for auditors and project managers.
Migrate with SPMT (Methods B and C)	Site Owner on the destination site (site collection administrator recommended)	Per destination site. SPMT does not require a tenant admin role.
Install the agent; agent / SPMT source access	Local Administrator (agent server); Read on the SMB share *and* NTFS Read & execute	Every source path. Use a dedicated service account, not a person's account. For permission preservation it must also read ACLs.
Run the cleanup script	NTFS Modify on the source folders	Run on or against the file server by an administrator.
Cutover (make share read-only, remove drive maps)	Local Administrator on the file server; Group Policy edit rights on the relevant OU	Change-window only.

GOOD PRACTICE · JUST-IN-TIME PRIVILEGE

Make SharePoint Administrator and Teams Administrator *eligible* assignments in Microsoft Entra Privileged Identity Management and activate them for the migration window with MFA and the change number as justification. Store the agent service account's password in your privileged-access vault and remove its share access after decommissioning.

2.2 Prerequisites checklist

- [ ] Approved change record and a migration wave plan (which share goes to which team, channel and folder).
- [ ] Destination teams and channels created and named in final form (Teams admin center, or `New-Team` / `New-TeamChannel` from the MicrosoftTeams module); each channel's **Files** tab opened once so its folder exists.
- [ ] A dedicated 64-bit Windows Server (or Windows 10/11) machine for the agent on the same LAN as the file server, with .NET Framework 4.6.2 or later, ample RAM and free disk for temporary packages (see Microsoft's current agent prerequisites page for figures).
- [ ] Outbound HTTPS 443 from the agent to Microsoft 365 and Azure storage endpoints; any TLS-inspecting proxy configured to bypass them.
- [ ] A service account with Read on every source path, plus its credentials for the agent installer.
- [ ] Entra Connect synchronising users, if user mapping or permission preservation is used.

- [ ] Storage quota on the tenant for the volume to be migrated.
- [ ] Agreed scope: one share (or top-level folder) per channel folder; stale data, installers, ISO and PST files go to archive instead; permission preservation off by default; read-only retention of 30–90 days for the old share.

CHAPTER 3

Method A · Migration Manager in the SharePoint Admin Center

This is the method used in the reference engagement and the one to prefer for anything bigger than a handful of folders. The flow is: confirm the agent, scan the source, remediate warnings, then create and run migration tasks.

STEP 1 Open Migration Manager and confirm the agent

1. Sign in to **admin.microsoft.com**, activate your PIM role, and select **Admin centers › SharePoint**.
2. In the SharePoint admin center select **Migration** in the left navigation.
3. Under **File shares**, select **View tasks** (on first use this button reads *Get started* and walks you through downloading the agent).
4. Confirm the banner shows *Agent connected*. If a warning bar says agents have warnings, open the **Agents** tab and resolve them before scanning.

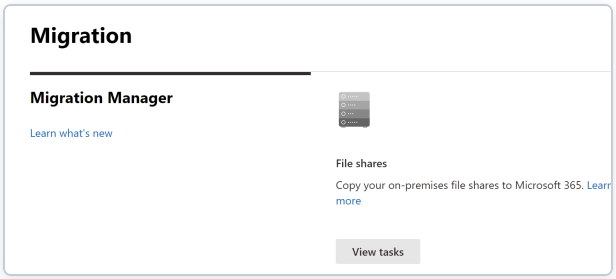


Figure 7. The Migration page. **View tasks** under *File shares* opens Migration Manager.

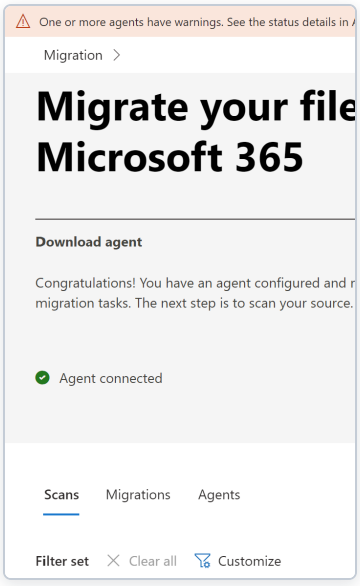


Figure 8. Migration Manager with *Agent connected* and the **Scans**, **Migrations** and **Agents** tabs.

NOTE · INSTALLING AN AGENT

Select **Download agent**, run the installer on the agent server as a local administrator, sign in with the SharePoint Administrator account when prompted, then enter the Windows service account that has Read on the shares. The installer offers a test against a UNC path; use it. The agent then appears on the **Agents** tab within a few minutes.

STEP 2 Scan each source path

1. On the **Scans** tab select **+ Add source path**.
2. Choose **Specify a single source path** and enter the full UNC path, for example `\\fileserver01\share\Projects`. For many paths choose **Upload a CSV file** with one UNC path per line.
3. Clear **Add all subfolders as source paths** unless you intend to migrate each subfolder to a different destination; leaving it checked creates one scan (and later one task) per subfolder.
4. Keep **Automatically scan content** selected and select **Add**.
5. Wait for the *Scan result* column. READY TO MIGRATE means no blocking issues. MIGRATION WARNINGS FOUND means items will fail or be skipped; remediate them as described later in this chapter before creating the task.

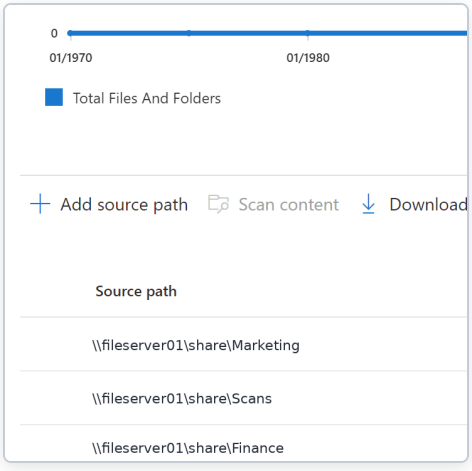


Figure 9. The Scans tab with **+ Add source path** and previously added sources.

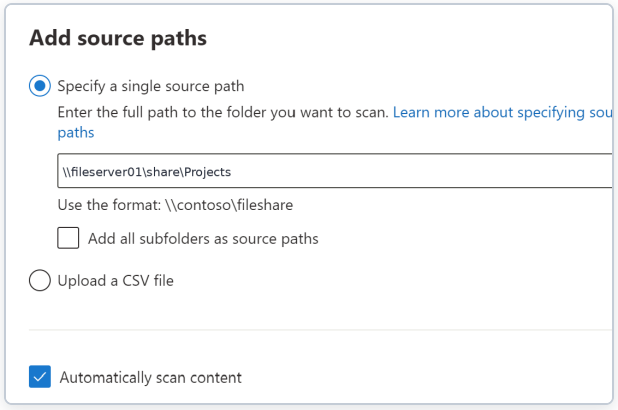


Figure 10. Add source paths: single path, subfolders unchecked, automatic scan on.

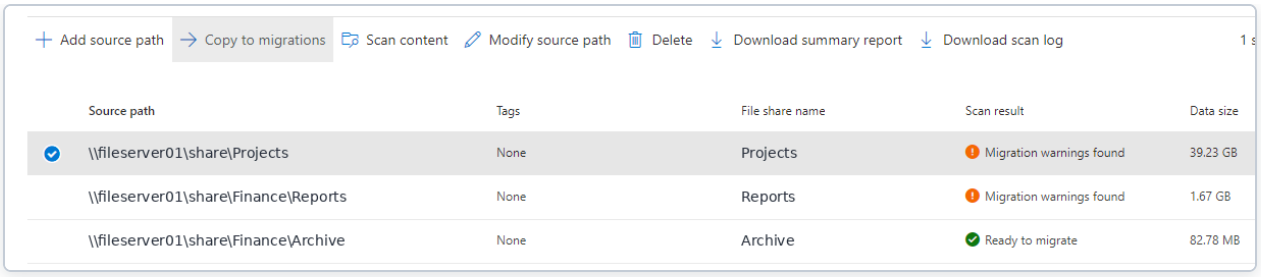


Figure 11. Scan results. Two sources show *Migration warnings found*; select one and use **Download scan log** to see which items are affected.

3.1 Remediating scan warnings

Remediation turns *Migration warnings found* into *Ready to migrate*. The input is the scan log; the output is a cleaner source and an evidence file that shows exactly what was changed.

Reading the scan log

1. On the **Scans** tab select the source with warnings and choose **Download scan log**. Save it to the project's evidence folder with a name such as `Projects - Scan Log.csv`.
2. Open it in Excel, select **Data › Filter**, and on the `ResultCode` column clear *(Blanks)*. What remains are the problem items, typically `INVALID_SHAREPOINT_NAME` and `ITEM_IS_EMPTY`.
3. Note the header names of the column containing the full UNC path of each item and of the result code column. The script below needs them.

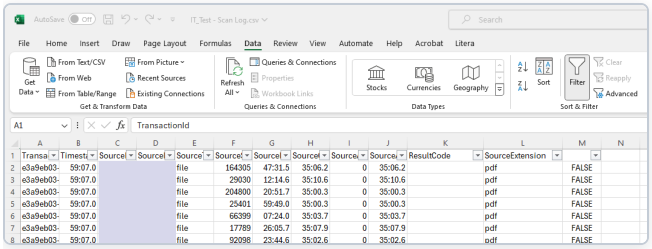


Figure 12. The scan log in Excel with filters on. One row per item; the path columns are hidden here.

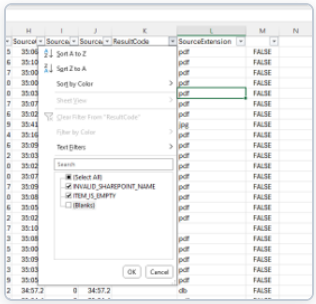


Figure 13. Filtering ResultCode to show only INVALID_SHAREPOINT_NAME and ITEM_IS_EMPTY.

From spreadsheet formula to a proper script

The original runbook generated commands with an Excel formula in a spare column and pasted them into an elevated console on the file server. It worked, but it had three

weaknesses: for any invalid name it deleted *every hidden file* beneath the parent folder, nothing was logged, and there was no dry run. The result looked like this:

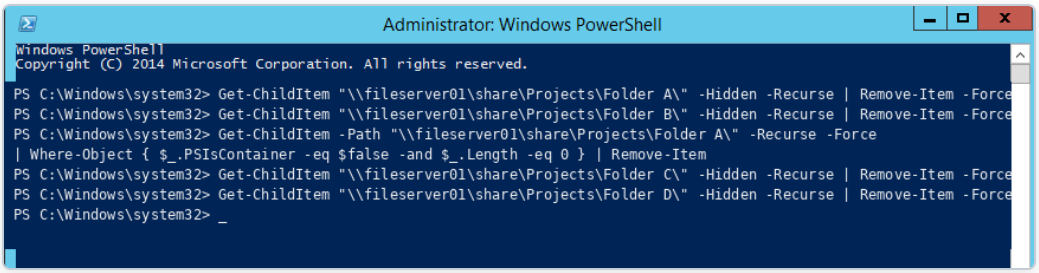


Figure 14. The legacy approach: formula-generated commands pasted into an administrator console. Effective, but unlogged and with no preview.

The template below replaces it. It reads the scan log directly, decides an action per item, supports `-WhatIf`, can move items to a quarantine share instead of deleting them, and writes a CSV evidence file.

INVOKE-PREMIGRATIONCLEANUP.PS1 · BASE TEMPLATE POWERSHELL

```powershell
<#
.SYNOPSIS  Remediate Migration Manager / SPMT scan-log warnings on the source file share.
.DESCRIPTION  Junk names are removed, fixable names renamed, zero-byte files removed, the rest flagged
              ManualReview. -QuarantineRoot moves instead of deleting. Always run with -WhatIf first.
.NOTES        Needs NTFS Modify on the source. Windows PowerShell 5.1 or PowerShell 7.
#>
[CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')]
param(
  [Parameter(Mandatory)] [ValidateScript({ Test-Path -LiteralPath $_ })] [string] $ScanLog,
  [string]   $PathColumn     = 'Source',      # header of the column holding the item's full UNC path
  [string]   $CodeColumn     = 'ResultCode',
  [string]   $QuarantineRoot,                 # e.g. \\fileserver01\quarantine\CHG0012345
  [string[]] $JunkNames      = @('desktop.ini','Thumbs.db','.DS_Store','~$*','*.tmp','*.lock'),
  [string]   $EvidenceFile   = ".\cleanup-evidence-$(Get-Date -Format yyyyMMdd-HHmmss).csv"
)
$ErrorActionPreference = 'Stop'

# ---- Stage 1: load and validate the scan log ------------------------------
$rows = Import-Csv -LiteralPath $ScanLog
$headers = $rows[0].PSObject.Properties.Name
foreach ($c in $PathColumn, $CodeColumn) {
  if ($headers -notcontains $c) { throw "Column '$c' not found. Headers are: $($headers -join ', ')" }
}
$targets = $rows | Where-Object { $_.$CodeColumn -in 'INVALID_SHAREPOINT_NAME', 'ITEM_IS_EMPTY' }
Write-Host "$($targets.Count) item(s) flagged in $ScanLog"

# ---- Stage 2: decide and act on each item ---------------------------------
$results = foreach ($r in $targets) {
  $path = $r.$PathColumn; $code = $r.$CodeColumn; $action = 'None'
  $status = if ($WhatIfPreference) { 'WhatIf' } else { 'Skipped' }
  try {
    $item = Get-Item -LiteralPath $path -Force
    $leaf = $item.Name
    $junk = $JunkNames | Where-Object { $leaf -like $_ }
    if ($code -eq 'ITEM_IS_EMPTY' -and -not $item.PSIsContainer -and $item.Length -eq 0) { $action =
'RemoveEmpty' }
    elseif ($code -eq 'INVALID_SHAREPOINT_NAME' -and $junk)                              { $action =
```

```
'RemoveJunk' }
    elseif ($code -eq 'INVALID_SHAREPOINT_NAME') {
      $new = ($leaf -replace '_vti_', '_vti-').Trim().TrimEnd('.')
      $action = if ($new -and $new -ne $leaf) { "Rename:$new" } else { 'ManualReview' }
    }

    if ($action -like 'Remove*') {
      $verb = if ($QuarantineRoot) { 'Move to quarantine' } else { 'Delete' }
      if ($PSCmdlet.ShouldProcess($path, $verb)) {
        if ($QuarantineRoot) {
          $rel  = $path -replace '^\\\\[^\\]+\\', ''            # drop the \\server\ prefix
          $dest = Join-Path $QuarantineRoot $rel
          New-Item -ItemType Directory -Path (Split-Path $dest) -Force | Out-Null
          Move-Item -LiteralPath $path -Destination $dest -Force
        } else {
          Remove-Item -LiteralPath $path -Force
        }
        $status = 'Done'
      }
    }
    elseif ($action -like 'Rename:*') {
      $newName = $action.Substring(7)
      if ($PSCmdlet.ShouldProcess($path, "Rename to '$newName'")) {
        Rename-Item -LiteralPath $path -NewName $newName; $status = 'Done'
      }
    }
  } catch { $status = "Error: $($_.Exception.Message)" }

  [pscustomobject]@{ TimeUtc = (Get-Date).ToUniversalTime().ToString('s'); ResultCode = $code
                     Path = $path; Action = $action; Status = $status; Operator =
"$env:USERDOMAIN\$env:USERNAME" }
}

# ---- Stage 3: evidence ------------------------------------------------------
$results | Export-Csv -LiteralPath $EvidenceFile -NoTypeInformation
$results | Group-Object Action, Status | Select-Object Count, Name | Format-Table -AutoSize
Write-Host "Evidence written to $EvidenceFile. Re-scan the source in Migration Manager."
```

EXAMPLE · DRY RUN, THEN QUARANTINE

INVOCATION — POWERSHELL

```
.\Invoke-PreMigrationCleanup.ps1 -ScanLog '.\Projects - Scan Log.csv' -WhatIf
.\Invoke-PreMigrationCleanup.ps1 -ScanLog '.\Projects - Scan Log.csv' `
    -QuarantineRoot '\\fileserver01\quarantine\CHG0012345' -Confirm:$false
```

Review the WhatIf output and especially every `ManualReview` line with the data owner before the real run.

NOTE · NAMES WINDOWS CANNOT OPEN

Files with trailing spaces or dots were usually created through APIs that bypass Win32 name normalisation, and ordinary cmdlets may fail on them with "path not found". Address them with the `\\?\UNC\fileserver01\share\...` prefix, which passes the name to the file system unchanged.

Finding long paths before the scan does

The scan reports paths that are too long, but it is quicker to find them up front and agree new folder names with the data owner. This one-liner measures each item's future SharePoint path:

FIND-LONGPATHS.PS1 POWERSHELL

```powershell
$src    = '\\fileserver01\share\Projects'
$prefix = 'https://contoso.sharepoint.com/sites/ContosoIT/Shared Documents/Infrastructure/Projects/'
Get-ChildItem -LiteralPath $src -Recurse -Force -ErrorAction SilentlyContinue | ForEach-Object {
    $rel = $_.FullName.Substring($src.Length).TrimStart('\') -replace '\\', '/'
    [pscustomobject]@{ TargetLength = ($prefix + $rel).Length; Path = $_.FullName }
} | Where-Object TargetLength -gt 400 | Sort-Object TargetLength -Descending |
    Export-Csv .\long-paths.csv -NoTypeInformation
```

STEP 3 Create and run a migration task

1. Open the **Migrations** tab and select **+ Add task**. (Alternatively select a clean scan and choose **Copy to migrations**.)
2. **Pick method**: **Single source and destination** for one share; **Bulk migration** to upload a CSV or JSON file listing many sources and destinations. Select **Next**.
3. **Select source**: enter the UNC path, for example `\\fileserver01\share\Projects`.
4. **Select destination**: choose **Teams** (or SharePoint for a plain site, OneDrive for personal data).
5. Select the destination team, then the channel. To land the data in a subfolder, select the arrow next to the channel, choose **Create folder**, enter a name such as `Projects` and select **Create**. Select **Next**.
6. **Settings**: enter a descriptive task name that includes the change number, choose **Run now** or **Run later**, and keep the default agent group. Leave **Preserve file share permissions** cleared unless the plan says otherwise. Review **All settings** for options such as file-age filters and excluded extensions.
7. Select **Run**. The task status moves through *Queued* and *In progress* to *Complete*, or to *Failed* with a **Download failure report** link. For a **delta pass**, select

the completed task and choose **Run** again (or **Schedule** it): only new or changed files are copied.

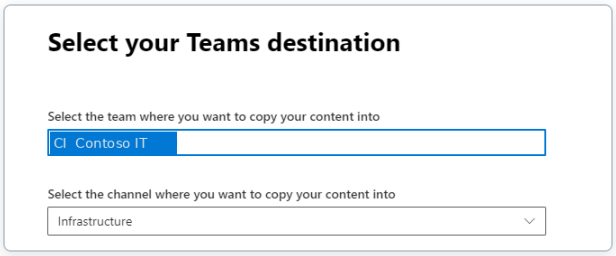


Figure 15. Select the team and channel.

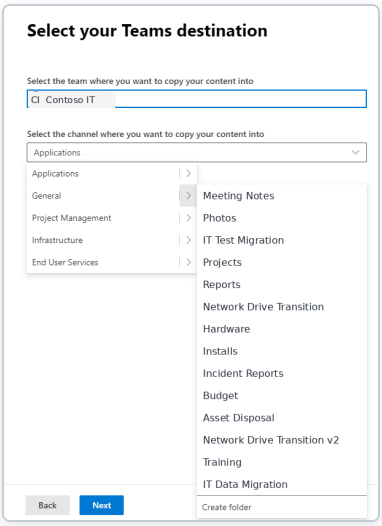


Figure 16. Existing folders in the channel; **Create folder** is at the bottom of the list.

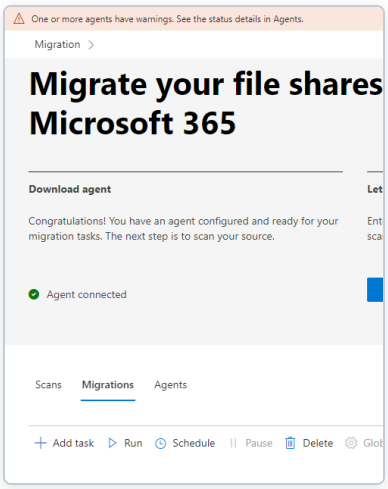


Figure 17. The Migrations tab with **+ Add task**, **Run** and **Schedule**.

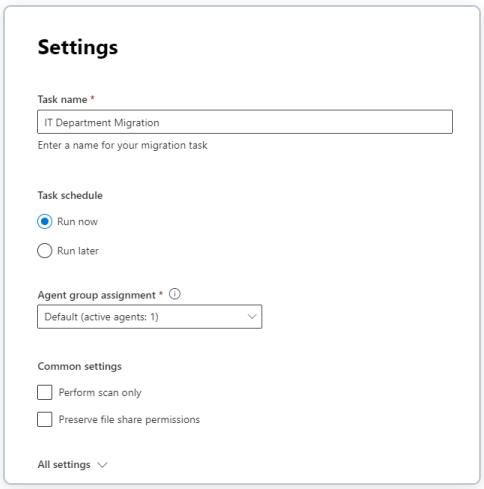


Figure 18. Task settings: name, schedule, agent group, scan-only and permission options.

CHAPTER 4

Methods B, C and D · SPMT and the Sync Alternative

The **SharePoint Migration Tool (SPMT)** is Microsoft's free desktop migration tool. It uses the same Migration API as Migration Manager but runs entirely on the machine where it is installed, under the signed-in user's identity. It needs no tenant admin role, only access to the destination site, which makes it handy for team owners and small projects.

METHOD B SPMT desktop wizard

1. Install SPMT on a 64-bit Windows machine near the file server (download link in the SharePoint admin center **Migration** page or Microsoft Learn). Sign in with an account that is Site Owner of the destination.
2. Select **Start your first migration** › **File share**.
3. Enter or browse to the source UNC path, for example `\\fileserver01\share\Projects`, and select **Next**.
4. Enter the destination team site URL (`https://contoso.sharepoint.com/sites/ContosoIT`), choose the *Documents* library and, under it, the channel folder (for example `Infrastructure`).
5. Name the migration and open **View all settings**. Recommended: keep *Migrate hidden files* off, turn *Preserve file share permissions* off, set an exclusion list (`.tmp, .lnk`), and keep incremental migration enabled.
6. Select **Migrate**. When finished, open the report folder and review the item-level failure report.
7. For a delta pass, run the *same* migration again from the same machine; SPMT copies only new and changed items.

> **NOTE · SCAN ONLY**
>
> SPMT has a *Scan only* setting that produces the same kind of pre-migration report as a Migration Manager scan. Use it when there is no agent.

METHOD C SPMT PowerShell base template (CSV-driven)

SPMT ships a PowerShell module, `Microsoft.SharePoint.MigrationTool.PowerShell`. The core pattern is three cmdlets: `Register-SPMTMigration` creates a migration session with its settings, `Add-SPMTTask` adds one source → destination pair, and `Start-SPMTMigration` runs every task in the session. Driving `Add-SPMTTask` from a CSV turns a wave plan directly into an executable change.

WAVE1-MAP.CSV TEXT

```
Source,TargetSiteUrl,TargetList,TargetFolder
\\fileserver01\share\Projects,https://contoso.sharepoint.com/sites/ContosoIT,Documents,Infrastructu
re/Projects
\\fileserver01\share\Apps,https://contoso.sharepoint.com/sites/ContosoIT,Documents,Applications
```

START-SHARETOTEAMSMIGRATION.PS1 · BASE TEMPLATE POWERSHELL

```
<#
.SYNOPSIS  Migrate file shares into Teams channel folders with the SPMT PowerShell module.
.NOTES     Run in Windows PowerShell 5.1 on a machine with SPMT installed (the module targets .NET
Framework).
           Account: Site Owner (site collection admin recommended) on each destination; Read on
each source.
           Register-SPMTMigration options vary by SPMT release: check Get-Help Register-
SPMTMigration -Full.
#>
[CmdletBinding(SupportsShouldProcess)]
param(
  [Parameter(Mandatory)] [ValidateScript({ Test-Path $_ })] [string] $MappingCsv,
  [Parameter(Mandatory)] [string] $ChangeRef,                    # e.g. CHG0012345
  [string] $ModulePath = 'Microsoft.SharePoint.MigrationTool.PowerShell',  # or full path to the
.psd1 in the SPMT install folder
  [switch] $ScanOnly,
  [switch] $PreservePermissions,
  [string] $EvidenceFile = ".\$ChangeRef-spmt-evidence.json"
)
$ErrorActionPreference = 'Stop'
Import-Module $ModulePath

# ---- Stage 1: validate the mapping ----------------------------------------
$map = Import-Csv -LiteralPath $MappingCsv
foreach ($row in $map) {
  if (-not (Test-Path -LiteralPath $row.Source)) { throw "Source not reachable: $($row.Source)" }
}

# ---- Stage 2: register the migration session ------------------------------
$cred = Get-Credential -Message 'Account with Site Owner rights on the destination sites'
$settings = @{
  SPOCredential                      = $cred
  Force                              = $true      # replace any existing session in this
PowerShell process
  ScanOnly                           = [bool]$ScanOnly
  PreserveUserPermissionsForFileShare = [bool]$PreservePermissions
  MigrateHiddenItems                 = $false
  SkipFilesWithExtension             = 'tmp,lnk'
}
if ($PSCmdlet.ShouldProcess('SPMT session', 'Register-SPMTMigration')) { Register-SPMTMigration
```

```
@settings }

# ---- Stage 3: one task per CSV row ----------------------------------------
foreach ($row in $map) {
  $target = "$($row.TargetSiteUrl)/$($row.TargetList)/$($row.TargetFolder)"
  if ($PSCmdlet.ShouldProcess("$($row.Source) -> $target", 'Add-SPMTTask')) {
    Add-SPMTTask -FileShareSource $row.Source -TargetSiteUrl $row.TargetSiteUrl `
                 -TargetList $row.TargetList -TargetListRelativePath $row.TargetFolder
  }
}

# ---- Stage 4: run (blocks until done; rerun later for a delta pass) -------
if ($PSCmdlet.ShouldProcess("$($map.Count) task(s)", 'Start-SPMTMigration')) {
  $start = Get-Date
  Start-SPMTMigration
  [pscustomobject]@{
    ChangeRef = $ChangeRef; Operator = $cred.UserName; StartUtc = $start.ToUniversalTime(); EndUtc
= (Get-Date).ToUniversalTime()
    ScanOnly  = [bool]$ScanOnly; Tasks = $map; Session = (Get-SPMTMigration | Out-String)
  } | ConvertTo-Json -Depth 4 | Out-File -LiteralPath $EvidenceFile
  Write-Host "Done. SPMT item reports are in the SPMT working folder; evidence in $EvidenceFile" -
ForegroundColor Green
}
```

EXAMPLE · SCAN, MIGRATE, THEN DELTA

INVOCATION — POWERSHELL

```
.\Start-ShareToTeamsMigration.ps1 -MappingCsv .\wave1-map.csv -ChangeRef CHG0012345 -WhatIf
.\Start-ShareToTeamsMigration.ps1 -MappingCsv .\wave1-map.csv -ChangeRef CHG0012345 -ScanOnly
.\Start-ShareToTeamsMigration.ps1 -MappingCsv .\wave1-map.csv -ChangeRef CHG0012345
# cutover night: same command again = incremental (delta) pass
```

NOTE · AUTHENTICATION

`Get-Credential` suits password sign-in. Where MFA or Conditional Access applies, use the modern-authentication sign-in your SPMT release supports.

Related · Migration Manager bulk file

For waves of many shares, **Bulk migration** takes a CSV with six columns and **no header row**: source path, source library and source subfolder (both blank for file shares), destination site URL, destination library, destination subfolder.

WAVE1.CSV — TEXT

```
\\fileserver01\share\Projects,,,https://contoso.sharepoint.com/sites/ContosoIT,Documents,Infrastruc
ture/Projects
\\fileserver01\share\Apps,,,https://contoso.sharepoint.com/sites/ContosoIT,Documents,Applications
\\fileserver01\share\Finance\Reports,,,https://contoso.sharepoint.com/sites/ContosoFinance,Document
s,General/Reports
```

4.1 Method D · Copying through OneDrive sync (not recommended)

It is tempting to sync the channel to a server with the OneDrive client and run Robocopy into the synced folder. It is included here as a contrast so that the trade-offs are understood and documented when someone proposes it.

ANTI-PATTERN FOR COMPARISON ONLY CMD

```
robocopy "\\fileserver01\share\Projects" "C:\Users\svc-migrate\Contoso\Contoso IT -
Infrastructure\Projects" ^
  /E /COPY:DAT /DCOPY:T /R:1 /W:1 /XF desktop.ini Thumbs.db ~$* *.tmp /LOG:C:\Temp\robocopy-projects.log
```

Compared with Migration Manager or SPMT, this approach stamps every file with the sync account as *Created by / Modified by*, often resets timestamps, runs through a single heavily throttled client, has no per-item reporting, and degrades beyond a few hundred thousand files. Worse, the synced folder lives on the server's local disk: if the disk fills or the client pauses, files exist only locally and may never reach SharePoint. Use it only for a few gigabytes when every supported tool is blocked, and verify counts afterwards.

CHAPTER 5

Cutover, Verification and Recovery

5.1 Pilot

Migrate one representative share for a small team first. Ask the pilot users to open, edit and co-author files, follow links from other documents, and sync a folder. Collect issues (broken links in spreadsheets, macros with hard-coded UNC paths, application files that should never have been on a share) and fix the plan before the bulk wave.

5.2 Cutover runbook

1. **T-5 days**: communicate the date, the new location, and that the old drive becomes read-only. Publish a one-page "where are my files" guide.
2. **T-1 day**: run a delta pass and confirm its duration fits the window.
3. **T-0, start of window**: make the source read-only at the share level so nothing changes during the final pass.

ON THE FILE SERVER (ELEVATED) POWERSHELL

```
Get-SmbShareAccess -Name 'Projects' | Export-Csv .\Projects-share-acl-before.csv -
NoTypeInformation
Revoke-SmbShareAccess -Name 'Projects' -AccountName 'CONTOSO\Domain Users' -Force
Grant-SmbShareAccess  -Name 'Projects' -AccountName 'CONTOSO\Domain Users' -AccessRight Read -
Force
```

4. Run the final delta (rerun the Migration Manager task, or rerun the SPMT script).
5. Verify (next section). Then remove the mapped drive: in the Group Policy that maps it, change the Drive Maps item's action to *Delete*, or remove it from the logon script.
6. Optionally deploy a `Sync` or `Add shortcut to My files` instruction for heavy users, and pin the channel for the team.
7. Keep the share read-only for the agreed retention period, then back it up and archive or decommission it.

WATCH OUT · SHORTCUTS AND LINKS

Desktop shortcuts, Office "recent files", Excel external links and Access databases still point at `\\fileserver01\...`. Read-only access keeps them working for viewing during the transition and makes the remaining dependencies visible.

5.3 Verification and evidence

Tool reports

- [ ] Task status *Complete* for every source in the wave
- [ ] Summary report: items scanned = migrated + skipped (by design)
- [ ] Failure report reviewed; every remaining failure explained
- [ ] Cleanup evidence CSV attached to the change
- [ ] Final delta finished inside the window

User-facing checks

- [ ] Random sample of files opens and co-authors in Teams
- [ ] Folder structure matches the source
- [ ] Modified dates and authors preserved
- [ ] Only team members can open the channel files
- [ ] Source share is read-only

For an independent count, compare the number of files on the source with the library's item count. On the source:

SOURCE COUNTS — POWERSHELL

```powershell
$src = '\\fileserver01\share\Projects'
$files = Get-ChildItem -LiteralPath $src -Recurse -File -Force -ErrorAction SilentlyContinue
[pscustomobject]@{ Files = $files.Count; GB = [math]::Round(($files | Measure-Object Length -Sum).Sum / 1GB, 2) }
```

In SharePoint, open the library and use **Settings › Site settings › Storage metrics** on the channel folder, or the Migration Manager summary report. Expect small differences for items the cleanup removed and for excluded extensions; record them.

5.4 Rollback

Because every method *copies*, the source is never at risk from the migration itself; only the cleanup step changes it, and quarantine mode makes that reversible.

Situation	Rollback action
Bulk copy wrong or incomplete	Delete the destination folder (it goes to the site recycle bin for 93 days), fix the mapping, run again.
Cutover abandoned	Restore share permissions from the exported ACL (`Grant-SmbShareAccess` with the original rights), re-enable the drive map, tell users to keep using the share.
Cleanup removed something needed	Move it back from the quarantine share (path structure is preserved), or restore from file-server backup.
Permissions wrong in Teams	Fix team membership or move content to a private channel; do not break inheritance on individual files.

5.5 Troubleshooting

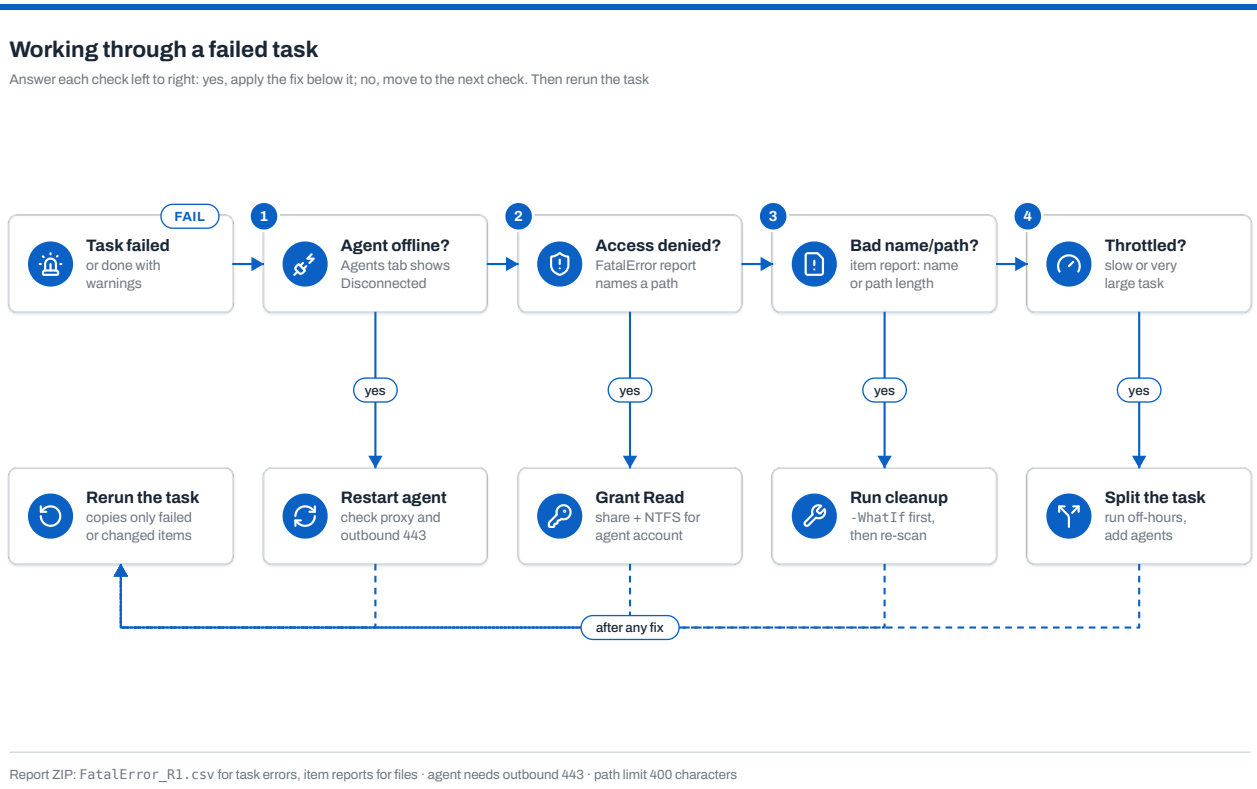


Figure 19. Most failures are an offline agent, a missing Read permission or an invalid name or path, so check those in that order before splitting or rescheduling a large task.

Symptom	Likely cause and fix
Status *Failed* with a failure report link	Download the report ZIP; open `FatalError_R1.csv` for task-level errors and the item reports for per-file errors. Fix, then rerun the task.
Agent shows *Disconnected* or warnings	Agent service stopped, password of the service account changed, or proxy blocking 443. Restart the service, update credentials, check proxy bypass.
"Access denied" to source	The agent account lacks share or NTFS Read on a subfolder with broken inheritance. Grant Read, rescan.
Channel missing as destination	Channel folder not provisioned yet; open the channel's Files tab once. Private channels must be chosen as their own site.
`INVALID_SHAREPOINT_NAME`	Reserved names, `~$` files, `_vti_`, leading or trailing spaces. Run the cleanup script.
`ITEM_IS_EMPTY`	Zero-byte files. Remove or quarantine with the cleanup script, or accept them as skipped.

Path too long	Target path over 400 characters. Migrate the subtree to a shallower folder or shorten names with the owner.
Very slow progress	SharePoint throttling or limited upload bandwidth. Add agents, split tasks, run outside business peaks.
Files show the migration account as author	Identity not found in Entra ID. Check Entra Connect sync or supply a user mapping file.

CHAPTER 6

Summary, Glossary and Review

6.1 Chapter summary

- Teams stores channel files in the team's SharePoint site; standard channels are folders in the Documents library, private and shared channels have their own sites.
- Migration Manager separates cloud control from an on-premises agent that reads shares and uploads through the Migration API.
- Scan first, remediate invalid names, empty files and long paths with a logged, `-WhatIf`-capable script, then re-scan.
- Use Migration Manager for scale, SPMT GUI for small jobs, and the SPMT PowerShell template for repeatable, CSV-driven waves.
- Bulk copy while users work, run deltas, make the source read-only, run the final delta and cut over.

Check your understanding

1. A user says "I uploaded the files to Teams." Where, physically, are those files stored for a standard channel and for a private channel?
2. Why does the Migration Manager agent need only outbound HTTPS, and what Windows rights does its service account need?
3. Name three things that produce `INVALID_SHAREPOINT_NAME` for content that already lives on an NTFS file share.
4. Why is preserving NTFS permissions usually a poor choice when the destination is a Teams channel?
5. Describe the sequence of actions in the cutover window and explain why the source is made read-only before the final delta.

6.2 Glossary

Migration Manager	Cloud-managed migration service in the SharePoint admin center that drives on-premises agents.
SPMT	SharePoint Migration Tool; free desktop tool and PowerShell module for migrations to Microsoft 365.
Delta (incremental) pass	A rerun that copies only items created or changed since the previous run.